L’authentification

0. Pré-requis

Créez un nouveau projet nommé « vitrine » dans lequel on va avoir deux entités :

- Une entité Categorie qui contient les champs suivants :
  - nom (string 255)
  - description (text, nullable = true)
  - slug (string 255)

- Une entité Produit qui contient les champs suivants :
  - nom (string 255)
  - description (text, nullable = true)
  - prix (float)
  - stock (integer)
  - image (string 255, nullable = true)
  - categorie (relation ManyToOne vers Categorie)

Faite votre migration, puis importez la Fixture que je vais vous donner ☺

Créez ensuite un controller pour chaque entité puis ces trois routes :

- **/produits** renvoie vers le fichier /produit/index.html.twig
- **/produit/{id}** renvoie vers le fichier /produit/detail.html.twig
- **/categories/{slug}/produits** renvoie vers le fichier /produit/index.html.twig avec les produits filtrés par catégorie

1. Configuration

Symfony et surtout son **MakerBundle** vont grandement nous faciliter la tâche ici puisqu’ils sont prévus pour gérer l’authentification des utilisateurs TRES efficacement.

Nous allons donc appeler le MakerBundle pour lui demander de créer les Entités correspondantes, **MAIS**

Le MakerBundle dispose d’une commande dédiée pour la gestion des utilisateurs, puisque c’est très fréquent. On ne va donc pas utiliser **make:entity** mais bien **make:user**

```
php bin/console make:user
```

Et on va répondre avec le choix par défaut pour toutes les questions que MakerBundle va nous poser.

On pourra à l'avenir rajouter des champs dans l'entity User qui a été généré si on le souhaite, par exemple un **nom**, puisque l'entité généré ne contient par défaut que des rôles, un mot de passe et un email. J'aime bien le faire perso, ça permet de gérer les utilisateurs un petit peu plus simplement, alors cette fois je lance bien **make:entity User** et j'ajoute le champ **nom**.

2. Générer le système de login

Symfony sait ce qu'il doit faire pour avoir une méthode d'authenfication forte, et il nous permet de la gérer facilement, encore une fois avec le **MakerBundle**.

Ici on va donc créer un formulaire de connexion avec la commande :

```
php bin/console make:security:form-login
```

On répond par défaut à toutes les questions posées et on se retrouve avec deux nouveaux fichiers :

- Un nouveau controller (SecurityController)
- Un nouveau template (login.html.twig)
- Et le fichier config/packages/security.yaml a été modifié automatiquement.

Si on va voir dans le fichier **security.yaml**, on peut y voir que le nom de l'Entité utilisateur ainsi que le chemin vers le formulaire de login on été copiés dans le fichier, ainsi que d'autres **règles** de fonctionnement :

- **password_hashers** — Symfony hash automatiquement les mots de passe, vous n'avez jamais à le faire manuellement.
- **providers** — indique à Symfony où chercher les utilisateurs, ici en base via l'email.
- **firewalls** — définit les règles de sécurité par zone de l'application. Le firewall dev exclut le Profiler de toute vérification. On va rajouter une règle **default_target_path** pour avoir une redirection automatique en cas de Login réussi, et on va également lui préciser quels sont les noms des champs du template qu'il doit traiter :

```yaml
form_login:
    login_path: app_login
    check_path: app_login
    enable_csrf: true
    default_target_path: app_admin_dashboard
    username_parameter: email
    password_parameter: password
```

*Attention : Il faut bien penser à accorder l'attribut **name** des champs dans le template également pour matcher avec les clés : username_parameter et password_parameter*

- **access_control** — On va décommenter la première ligne : toutes les URLs commençant par /admin nécessitent ROLE_ADMIN. C'est une règle globale, simple et efficace.

```
access_control:
    - { path: ^/admin, roles: ROLE_ADMIN }
    # - { path: ^/profile, roles: ROLE_USER }
```

3. Mettre à jour la base

```
php bin/console make:migration
```

```
php bin/console doctrine:migrations:migrate
```

4. Créer et Charger les Fixtures

En général on sépare les Fixtures des Utilisateurs du reste de la logique métier du site, et c'est ce qu'on va faire ici. On va d'abord créer la Fixture UserFixtures

```
php bin/console make:fixtures UserFixtures
```

Puis on va y mettre deux utilisateurs types : Un utilisateur non admin, et un utilisateur Admin. Copier cette fixture puis chargez-la :

```php
<?php
// src/DataFixtures/UserFixtures.php
namespace App\DataFixtures;

use App\Entity\User;
use Doctrine\Bundle\FixturesBundle\Fixture;
use Doctrine\Persistence\ObjectManager;
use Symfony\Component\PasswordHasher\Hasher\UserPasswordHasherInterface;

class UserFixtures extends Fixture
{
    public function __construct(
        private UserPasswordHasherInterface $hasher
    ) {}

    public function load(ObjectManager $manager): void
    {
        // Utilisateur admin
        $admin = new User();
        $admin->setEmail('admin@vitrine.fr');
        $admin->setNom('Administrateur');
        $admin->setRoles(['ROLE_ADMIN']);
        $admin->setPassword($this->hasher->hashPassword($admin, 'admin1234'));
```

```
        $manager->persist($admin);

        // Utilisateur standard
        $user = new User();
        $user->setEmail('user@vitrine.fr');
        $user->setNom('Utilisateur');
        $user->setRoles([]);
        $user->setPassword($this->hasher->hashPassword($admin, 'user1234'));
        $manager->persist($user);

        $manager->flush();
    }
}
```

```
php bin/console doctrine:fixtures:load
```

5. Créer le back-office

Maintenant qu'on a nos templates de prêts et nos utilisateurs en base, nous allons créer une partie **back-office** pour gérer le site.

Pour rappel, le Back Office est une partie **non accessible au public**, protégée par une identification obligatoire, et qui permet en général d'administrer le contenu du site, voire plus ! Gérer les comptes utilisateurs, collecter des statistiques, etc…

Nous allons donc créer un controller dédié et le modifier légèrement :

```
php bin/console make:controller AdminController
```

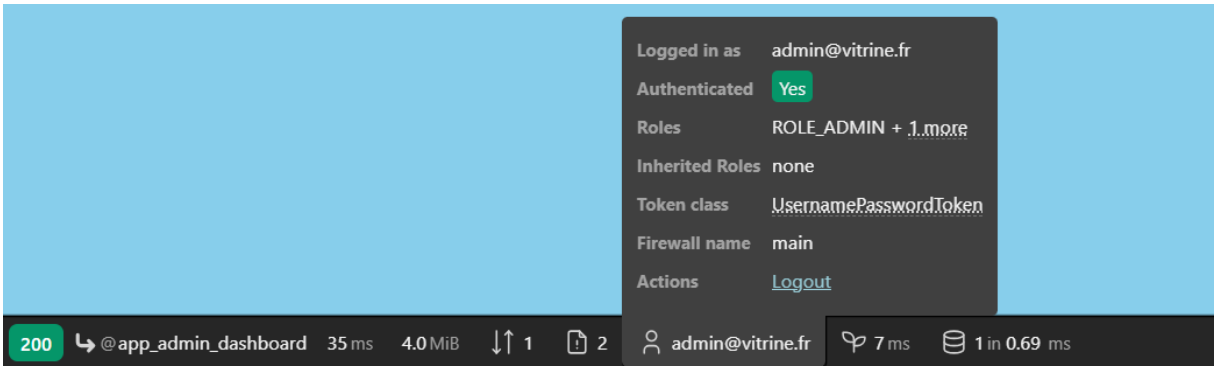
```
<?php
// src/Controller/AdminController.php
namespace App\Controller ;
use Symfony\Bundle\FrameworkBundle\Controller\AbstractController;
use Symfony\Component\HttpFoundation\Response;
use Symfony\Component\Routing\Attribute\Route;

// Ce décorateur sur le nom de la classe permet d'ajouter le préfix /admin avant chaque route
// Ça nous permet de coller avec la règle que l'on a défini dans security.yaml sans rien rajouter
#[Route('/admin', name: 'app_admin_')]
class AdminController extends AbstractController
{
    #[Route('/dashboard', name: 'dashboard')]
    public function dashboard(): Response
    {
        return $this->render('admin/dashboard.html.twig');
    }
}
```

6. Étape de vérification

Normalement, si vous avez tout bien fait et que vous avez :

- Une route app_admin_dashboard
- Un template admin/dashboard.html.twig
- Un utilisateur correspondant en base de données (Vous pouvez vérifier avec : `php bin/console doctrine:query:sql "SELECT * FROM user"`)
- Les redirections bien construites dans **security.yaml**

Vous devriez normalement être redirigé vers votre template dashboard.html.twig avec cette petite nouveauté dans la barre détat de Symfony, en bas de l'écran :



C'est que tout est bon ☺ On va pouvoir poursuivre le cours d'après en mettant en place des CRUDs en Admin, mais d'abord : On va rendre tout ça joli un peu ! Pratiquons l'intégration au fil de l'eau, je veux des templates sympa, des menus pour naviguer entre les différentes pages, Une liste des catégories qui permet d'arriver sur les pages filtrées et une vue détail de chaque produit.